

5. Out of Boundary Paths

Aim

To write a Python program to find the **number of paths that move out of the boundary of a grid within a given number of moves** using Dynamic Programming.

Algorithm

1. Start with a grid of $m \times n$.
2. Given a starting position (row, col) and maximum number of moves.
3. If the current position is outside the grid, return 1.
4. If no moves are left, return 0.
5. For each move, try moving **up, down, left, and right**.
6. Store already calculated results using Dynamic Programming.
7. Add the number of paths from all four directions.
8. Return the total number of paths.

Source Code

```
def find_paths(m, n, maxMove, row, col):
```

```
    dp = {}
```

```
    def dfs(r, c, moves):
```

```
        if r < 0 or r >= m or c < 0 or c >= n:
```

```
            return 1
```

```
        if moves == 0:
```

```
            return 0
```

```
        if (r, c, moves) in dp:
```

```
            return dp[(r, c, moves)]
```

```
        paths = (
```

```
            dfs(r + 1, c, moves - 1) +
```

```
            dfs(r - 1, c, moves - 1) +
```

```

        dfs(r, c + 1, moves - 1) +
        dfs(r, c - 1, moves - 1)
    )
    dp[(r, c, moves)] = paths
    return paths

return dfs(row, col, maxMove)

m = 2
n = 2
maxMove = 2
row = 0
col = 0

print("Number of paths:", find_paths(m, n, maxMove, row, col))

```

Output

The screenshot shows a code editor with a file named 'main.py'. The code defines a function `find_paths(m, n, maxMove, row, col)` that uses a recursive `dfs` function with memoization to count the number of paths from `(row, col)` to `(m, n)` with a maximum of `maxMove` steps. The output of the script is 'Number of paths: 6'.

```

1 def find_paths(m, n, maxMove, row, col):
2     dp = {}
3
4     def dfs(r, c, moves):
5         if r < 0 or r >= m or c < 0 or c >= n:
6             return 1
7
8         if moves == 0:
9             return 0
10
11        if (r, c, moves) in dp:
12            return dp[(r, c, moves)]
13
14        paths = (
15            dfs(r + 1, c, moves - 1) +
16            dfs(r - 1, c, moves - 1) +

```

Output: Number of paths: 6

Complexity Analysis

There are $m \times n \times \text{maxMove}$ possible states.

- **Time Complexity:** $O(m \times n \times \text{maxMove})$
- **Space Complexity:** $O(m \times n \times \text{maxMove})$

Result

Thus, the **Out of Boundary Paths** problem was successfully implemented using **Dynamic Programming**, and its time complexity is **$O(m \times n \times \text{maxMove})$** .